ICE Lab LabVIEW Programming

7/1/2026

***Rev.* 1**

ABSTRACT:

This document describes the LabView Client Template project ("Client_Template_MODBUS.lvproj"), programmed to support the Instrumentation and Control Laboratory at Murdoch University as part of the School of Engineering and Energy. The document gives full coverage of client-side communication enabling full use of engineering stations ES1-ES8 and is intended to serve as an all-encompassing description of all programming function for the purposes of technical understanding, troubleshooting, and future development.

Contents

List of Figures

List of Tables

Document Control and Revision History

Table 1: Document Control and Revision History

Revision	Date	Author(s)	Reviewed by	Comments/Details
1	2026	Nicholas O'Keeffe		Initial Commit.

1.0 Purpose & Scope

The purpose of this document, as stated in the abstract, is to give full coverage of client-side communication enabling full use of engineering stations ES1-ES8 in the Murdoch university Instrumentation and Control Engineering (ICE) Laboratory. Further, it is intended to serve as an all-encompassing description of all programming function for the purposes of technical understanding, troubleshooting, and future development.

In proceeding sections, the full project philosophy, topology, master program, and supporting functions (subVI's) are described. This aim is to give the intent of the program design, as well as birds-eye down to granular descriptions of the program flow and function.

Excluded from the scope of this document is:

- **MATLAB CLIENT INTERFACE –** The MATLAB client interface is intended to support communication with 3rd party applications to enable control system design in cases where the advanced nature of the strategy becomes too cumbersome to be integrated with LabVIEW. Any MODBUS/TCP compliant application is supported; however, a MATLAB template has been developed and is available within the client template package. This is covered in a separate document, "ICE-1002 ICE Lab MATLAB Programming".
- **MODBUS/TCP Server Design and Support –** Server-side programming and communication, and hardware level process integration are not covered here. This is covered in a separate vendor supplied document. At the time of the production of REV 1 of this document, this was titled, "ICE_Lab_User_Manual_REV2_2". A copy of this is supplied within the client template package, however, care should be taken to ensure this is up to date, as work on ICE lab upgrades had not been finalised during the production of client template support documentation.

2.0 Programming Philosophy

The Murdoch university ICE lab is used for the practical experimentation of the theory-side of the Industrial Automation and Control Engineering major as part of the school of engineering and energy. While the lab requires programming for control systems integration with physical processes, the emphasis is on theoretical control systems. Ideally programming requirements for experimentation are to be minimised for this process.

Previously, lab utilised LabVIEW for full-stack integration of physical hardware with control design capable, student designed programming. This was intended to allow ongoing modification and upgrades of facility supporting software to be carried out by students, who over the course of relevant engineering majors, spent significant time working with LabVIEW. This philosophy had flaws, particularly surrounding reliability and system robustness.

As hardware reached end of life. An increase in student numbers and growing concerns regarding reliability prompted the upgrade of process integrating hardware and the change of operating philosophy to limit student influence on critical systems. With this change, the requirements for student operable programming across the whole system was no longer an issue, and more practical solutions could be employed. This change in philosophy lead to the new server design which integrates the existing 4-20mA interfacing hardware with ADAMs RTUs, each communicating with a MODBUS/TCP server hosted by a Linux machine.

With the server side changes, client-side changes where required to enable communication with the new server. Given student and staff experience with the pre-existing system, the primary philosophy of design for the package was the minimisation of visual and functional changes to the client side of the control hierarchy interacted with by the students. As such, some programming may appear redundant or superfluous, however, this was all intended to make the user experience functionally unchanged from the old programming environment. Some quality-of-life improvements have been made, but in its current state, no instruction would be required for a student, capable of using the previous system to adapt to the revised MODBUS/TCP based system.

3.0 Programming

The project is broken into multiple programs, functions, and LabVIEW specific subVI's. This section covers these items sequentially:

1. **User Interface –** Front facing program control – acts as human machine interface – user configurable.
2. **Project File – "**Client_Template_Modbus.lvproj" – LabVIEW program management file. Manages execution of main script and organisation of supporting subVI's.
3. **Main Script –** "Client_Template_MOD.vi" – Main LabVIEW program, manages all control of physical hardware.
4. **Supporting Programming –** Made of subVI's supporting critical functionality of the main script.

3.1 User Interface

The LabVIEW client template is partitioned into six sections (Figure 1). All sections are discussed below (Figure 1, i.-v.) with the exception of scratchpad monitoring and control (Figure 1, vi.), which is discussed in the document "ICE-1002 ICE Lab MATLAB Programming".

A. ES analog inputs/outputs (Figure 1 i., Figure 2)

This section corresponds to the monitoring of selected ES panel analog inputs, and the control of selected ES panel analog outputs. Values are updated on a per-step basis. Specific inputs/outputs can be selected with the enumerated control elements below vertical bars. Additional inputs/outputs can be added utilising the LabVIEW back panel. ***NOTE:*** *Always ensure there are not multiple instances of an input or output selected as this can cause unintended system behaviour.*

B. Online plotting of back panel configured numeric variables (Figure 1 ii., Figure 3)

This section corresponds to the monitoring of numeric variables configured in the LabVIEW back panel. Monitoring is performed over time. By default, the plot gives the values on configured analog inputs/outputs described in the previous sub-section, however, these can be changed/additional variables can be added in the LabVIEW back panel.

C. ES digital inputs/outputs (Figure 1 iii., Figure 4)

This section corresponds to the monitoring of selected ES panel digital inputs, and the control of selected ES panel digital outputs. Values are updated on a per-step basis. Specific inputs/outputs can be selected with the enumerated control elements to the left of indicators/controls. Additional inputs/outputs can be added utilising the LabVIEW back panel. ***NOTE:*** *Always ensure there are not multiple instances of an input- or output-channel selected as this can cause unintended system behaviour.*

D. ES panel communication and program telemetry (Figure 1 iv., Figure 5)

This section corresponds to the selection of ES, in this section program/communication telemetry can also be configured and monitored, this includes:

1. **Working indicator –** A Boolean indicator that flashes with program cycling, intended purely for visual confirmation that the program is operating.
2. **dt selection –** User selected target sample time, 200ms by default, program will attempt any selected value, however server update speed is capped at 150ms, so any sample time less than this value is silly.
3. **Time since start –** Real time since program started in seconds.
4. **Program stop button –** Halts program execution, this is how program should always be stopped, as stopping via LabVIEW tool bar can cause system errors.
5. **System error feedback –** Will display descriptions and codes for any errors encountered by the system during operation.

E. Online system logging (Figure 1 v., Figure 6)

This section corresponds to the embedded logging functionality of the program. Two interactive fields are included:

1. **Logging? –** Will begin logging of data when given a rising edge (i.e.: when Boolean toggle becomes true).
2. **Logging File Name –** The name the log file will take in the pre-specified file-directory path when logging begins/completes

The project is defined such that, when a log is created, it is automatically saved to the “Labview_Logging” file, inside the main project file. Logging and be stopped and started at any time, and as many times as desired. This is supported by appending the date and time to the user specified file name when generating a new log. For further information, including specification of how to add extra data to log file during logging, see section 3.4.2.

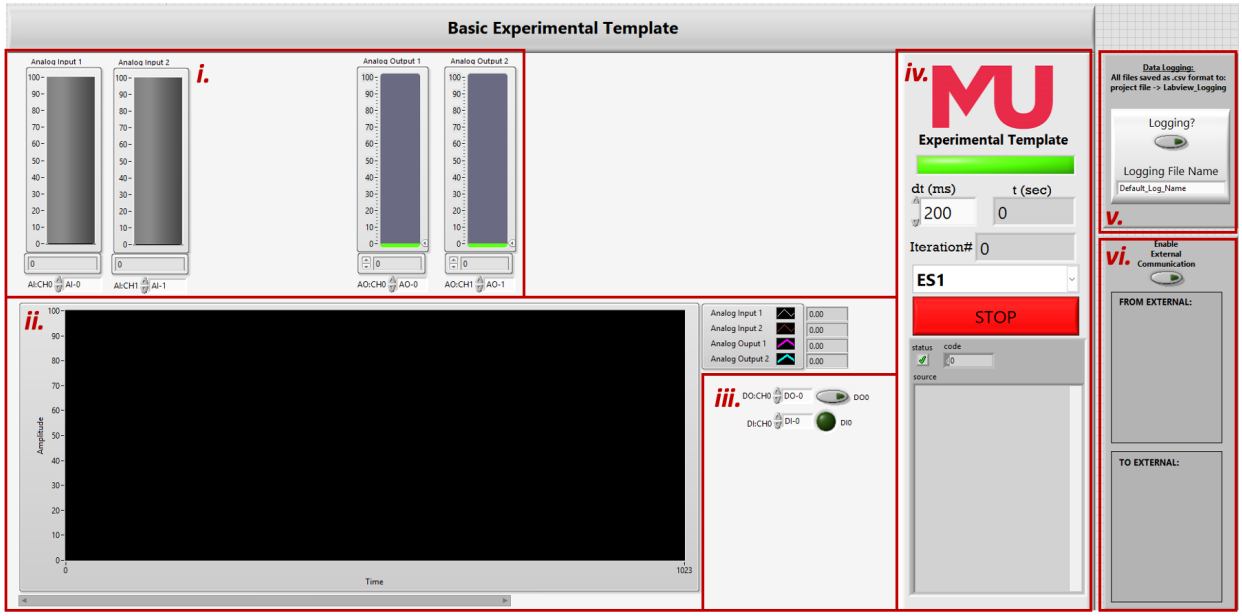


Figure 1: The LabVIEW Client Template front panel, partitioned sections correspond to: i. ES analog inputs/outputs (Figure 2), ii. online plotting of back panel configured numeric variables (Figure 3), iii. ES digital inputs/outputs (Figure 4), iv. ES panel communication and program telemetry (Figure 5), v. online system logging (Figure 6), vi. scratchpad monitoring (Figure 7).

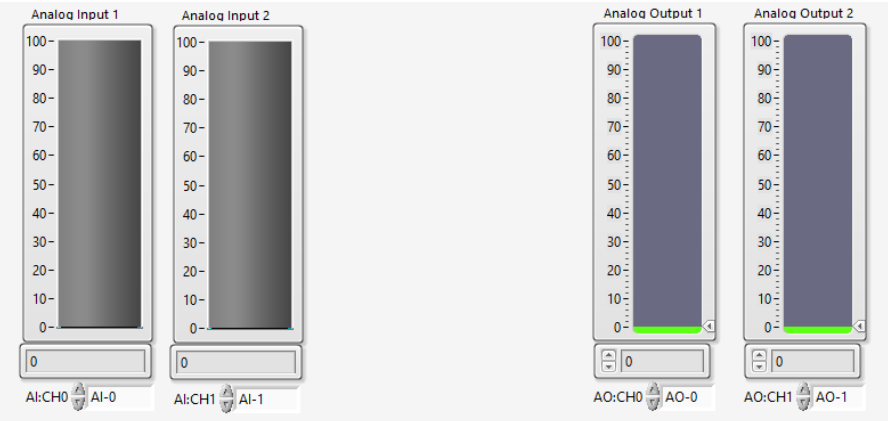


Figure 2: Portion of LabVIEW Client Template front panel for step wise monitoring and control of configured ES analog inputs and outputs.

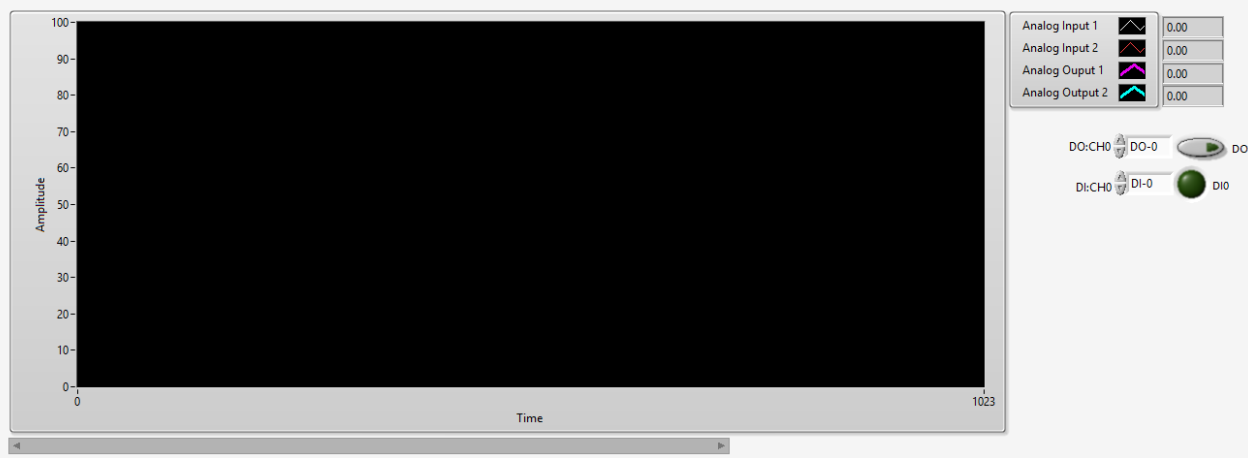


Figure 3: Portion of LabVIEW Client Template front panel for online plotting of any user configured program variables, these can correspond to ES panel analog inputs and outputs, or any other numeric signals generated in the program.

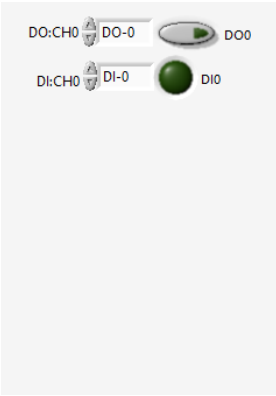


Figure 4: Portion of LabVIEW Client Template front panel for stepwise monitoring and control of configured ES digital inputs and outputs.

Figure 5: Portion of LabVIEW Client Template front panel for ES connection configuration and monitoring, and operation telemetry.

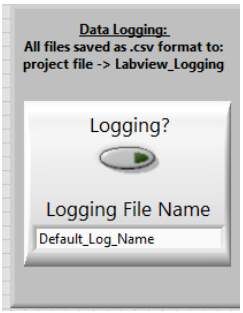


Figure 6: Portion of LabVIEW Client Template front panel for configuration and control of data logging. When active back panel configured logged variables are written to an auto-generated .csv file and the file is per-step updated.

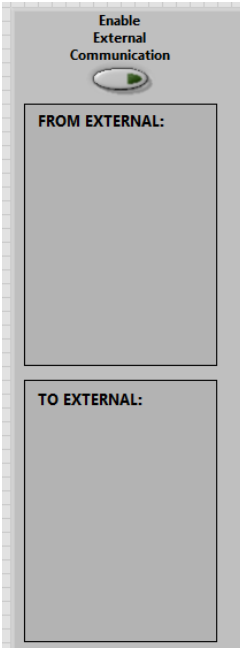


Figure 7: Portion of LabVIEW Client Template front panel for scratchpad interface monitoring. This is for advanced users and not required for typical use. The scratchpad interface is not discussed in this document, for full description see document "ICE-1002 ICE Lab MATLAB Programming".

3.2 Project File

LabVIEW uses project files to facilitate modular programming. Calling any subVI's from the main script requires specified file paths for any required functions. By storing all scripts and subVI's in a project file this can all be implicitly managed. As such, **the project file should always be opened first, and the main script launched in LabVIEW in the opened project.**

3.3 Overall Programming Flow

This section will cover the main script, it broadly covers each step of program execution to facilitate and maintain communication with the MODBUS/TCP server and consequently, interface with the physical process hardware. Only broad steps will be covered, with the function of any supporting subVI's covered in section 3.5. A flow diagram is given by Figure 8, and an image of the main communication loop is given by Figure 9.

A. Establish communication with server.

User management is manually controlled with the use of the gate containing holding register. If this is clear (= 0) then the panel is clear, and free to use. With the user selected engineering station (ES), the "ConnectToPannelAddressSpace.vi" SubVI checks the value of the gate and halts the program if the ES is in use. Otherwise, the local computer ID (derived from the assets IP) is written to the gate, signifying that the ES is in use. Once permission to connect is established, an additional check is performed to ensure server communication with the ES associated field I/O is healthy. This is performed by monitoring a server updated watchdog for 10 cycles before beginning operation. During healthy operation the watchdog should change at each cycle, if, following 10 reads of the register holding the watchdog, this is the case,

the program is allowed to continue, otherwise an error is returned, and the program is halted. The rationale for implementation in this way, instead of cyclically rechecking the watchdog during normal operation is threefold. Firstly, additional read operations add points of failure to the program – undesirable given robustness as a motivating factor for the system update – secondly, the additional server read/writes have the potential to bloat the network, potentially increasing minimum sample time. Finally, because it was the last thing I added, and if I did it the right way I would have had to revise all the SubVIs and review a metric shit tonne of documentation and testing.

B. (While stop conditions not met) Maintain connection with server.

The gate will automatically zero if no user has written to it in the timeout period. To allow continued communication, the user ID is rewritten at each cycle of the main loop with the "MaintainConnectionToPannelAddressSpace.vi" SubVI.

C. (While stop conditions not met) Read from input registers, holding registers, and discrete inputs.

Three subVI's manage reading values from the server:

1. "ReadFromPannelAddressSpace.vi" – Reads input registers, holding registers, and discrete inputs from server. Returns three arrays, an 8x1 16-bit UINT array of input register values, an 8x1 16-bit UINT array of holding registers, corresponding to the scratchpad input registers – these registers have no physical function and are used to allow the transfer of information between LabVIEW and 3rd party applications – and an 8x1 Boolean array of discrete inputs.
2. "AnalogInput.vi" – Takes an 8x1 16-bit array, and a specified address (0-7) and returns the value at the position of the array of the specified address, after conversion to a 2-point precision floating point number.
3. "DigitalInput.vi" – Takes an 8x1 Boolean array, and a specified address (0-7) and returns the value at the position of the array of the specified address.

Note: "AnalogInput.vi"'s and "DigitalInput.vi"'s can be daisy-chained together to read from multiple channels.

D. (While stop conditions not met) Write to holding registers, and discrete coils.

Three subVI's manage writing values to the server:

1. "AnalogOutput.vi"—Places a floating-point number, performs conversion to a 16-bit UINT, and places it into an 8x1 array in a user specified position.
2. "DigitalOutput.vi" – Places a Boolean value into an 8x1 array in a user specified position.
3. "WriteToPannelAddressSpace.vi" – Writes holding registers, and discrete coils to server. Accepts three arrays, an 8x1 16-bit UINT array of holding register values associated with the physical panel analog outputs, an 8x1 16-bit UINT array of holding register values associated with the scratchpad output registers, and an 8x1 Boolean array of discrete coil values associated with the physical panel digital outputs.

Note: "AnalogOutput.vi"'s and "DigitalOutput.vi"'s can be daisy-chained together to write to multiple channels.

E. Clear registers and disconnect from server

To keep the panel in a safe state, all holding registers and coils associated with physical panel outputs must be cleared. Once the panel has been made safe, the connection is cleared by clearing the holding register associated with the gate. Both aforementioned tasks are managed by the SubVI, "DisconnectFromPannelAddressSpace.vi".

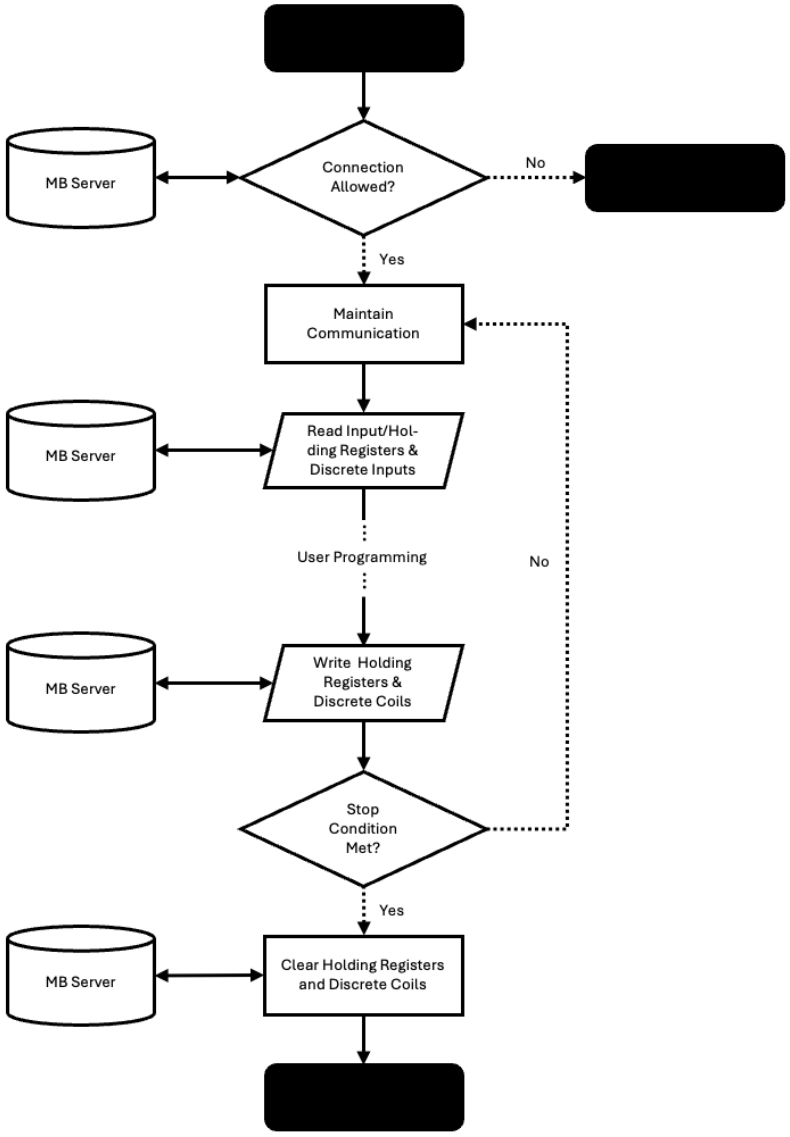


Figure 8: The overall program flow of main client template program. ***Note: Diagram is simplified, and only broad steps are presented.***

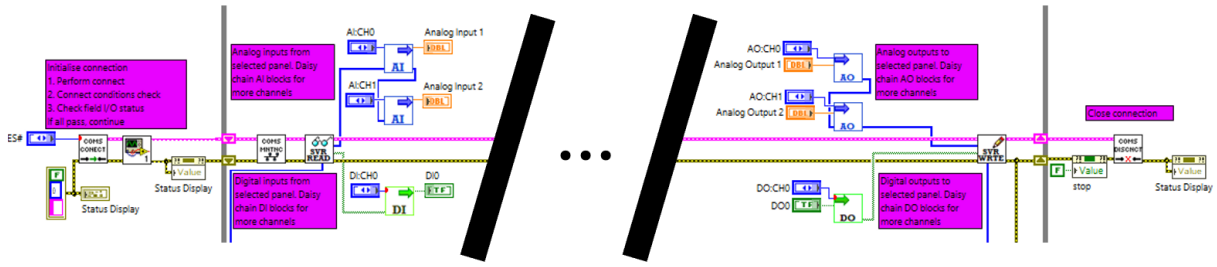


Figure 9: The primary communication loop for client-side engineering station control and monitoring of ICE lab physical processes.

3.4 Supporting Functionality

3.4.1 Communication With 3rd Party Applications

As previously stated, communication with 3rd party applications, most notably MATLAB, is not covered in this documentation and is instead covered in the separate document, "ICE-1002 ICE Lab MATLAB Programming". However, for completeness, a figure of the portion of the main program associated with this data transfer is given in Appendix B.

3.4.2 Native Inbuilt Logging

To support experimentation, users require the capability to collect long term data of experimental runs. Data is exported in the form of ".csv" files that can later be interpreted by applications like excel and MATLAB. Data collection is performed by the SubVI, "arbitrarySessionLog.vi". The SubVI takes an arbitrary width, 1-Dimensional array of type string, and an arbitrary width, 1-Dimensional array of type double. The only constraint on input arrays is that they must be the same width due to requirements of later concatenation for file formatting. The SubVI has a Boolean input which begins logging, this can be stopped and started arbitrarily with each reinitialisation generating a unique ".csv" file. The SubVI also has a string input which specifies the generated file name, "Default_File_Name" by default; to support arbitrary reinitialisations of logging creating unique files, the date and time are appended to this name before the file is generated.

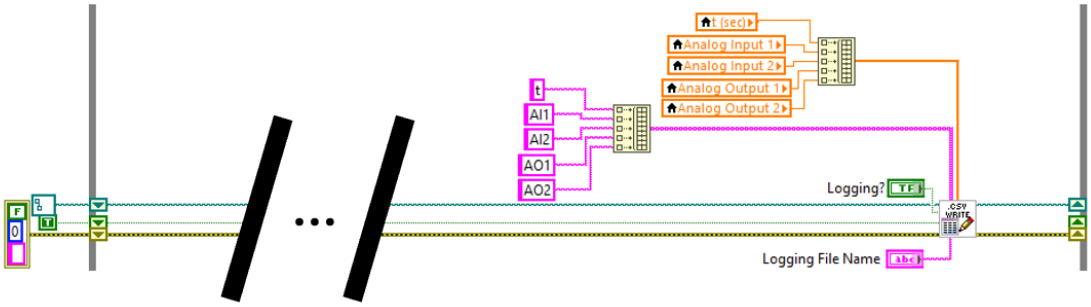


Figure 10: User logging loop. Activated by Boolean control on front-facing interface, system auto-creates ".csv" file based on user selected document title and time stamp.

3.4.3 Online Plotting

To enable online visual feedback of experimentation, plotting is included by default. The client template is preconfigured with four local variables that are concatenated into a waveform variable and fed into the standard LabVIEW waveform chart VI. Local variables can be changed easily or can be removed and replaced with lines directly wired into any pre-existing line in the program. Additional trends can be added by inserting the desired variables into the array that feeds the waveform chart.

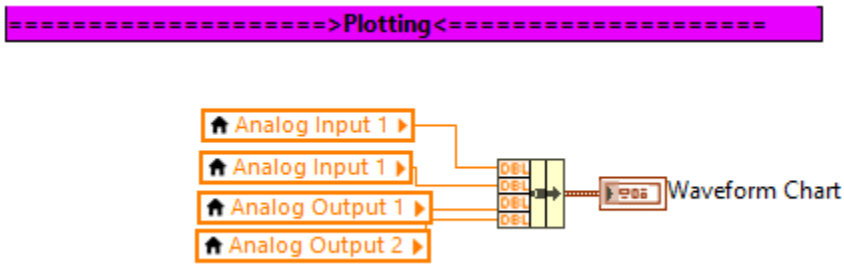


Figure 11: User defined plotting, by default four local variables are concatenated into waveform variable and plotted with default waveform chart block.

3.5 SubVI's

This section covers the functionality and working of all custom project SubVI's. For brevity, images of SubVI front-, and back-panels are available in Appendix A.

3.5.1 "AnalogInput.vi"

A. Function

Returns floating point value of desired physical panel analog input OR scratch input register. Can be daisy-chained to allow multiple reads of same input array.

B. Inputs

1. Desired read array offset – in the case of physical panel AI, this can be enumerated or int 0-7 – datatype INT
2. Array of integer values – can be any array, but AI is used in this project to read either input registers or scratch input registers – data type nx1 array of INT

C. Outputs

1. Value of scalar at array offset given by input – data type DBL
2. Identical array from SubVI input

D. Working

1. Index given by input 1 read from array given by input 2
2. Scalar index scaled from 0 – 10000 to 0 – 100.00 and passed to output 1
3. Input 2 passed directly to output 2

3.5.2 "AnalogOutput.vi"

A. Function

Inserts analog value into 8x1 array of values to be written to register by write SubVI. Can be used to construct arrays for holding registers associated with physical analog outputs or scratch registers. If the 8x1 input array of values is unwired, the output array will be initialised as 8x1 array of scalar values =0. Can be daisy-chained together to allow multiple writes to the same array.

B. Inputs

1. Desired write array offset – in the case of physical panel AO, this can be enumerated or int 0-7 – datatype INT
2. Analog output value – data type DBL
3. 8x1 array of values to insert output into – data type INT

C. Outputs

1. 8x1 array of values to pass into next "AnalogOutput.vi" SubVI OR write to MODBUS/TCP server – updated array given by input 3 – data type INT

D. Function

1. Analog value given by input 2with data type DBL is converted to data type INT with the sequence:

$$\text{int32}\left(\text{abs}\left(\text{round}(in * 100)\right)\right)$$

2. Converted INT is inserted into array given by input 3 at position given by input 1 and passed to output 1

3.5.3 "arbitrarySessionLog.vi"

A. Function

Creates ".csv" file and continuously updates generated file at each cycle of main program loop. Logging can be restarted an arbitrary number of times and at each reinitialisation a unique file will be generated. File name can be set by "Logging File Name" input. Generated files are saved to file path "<project file location>\Labview_Logging\...".

NOTE: SubVI can generate nx1, with arbitrary n dimension logs BUT "Column Names", and "Log Values" SubVI inputs must have same dimension.

B. Inputs

1. File path input – used to maintain generated file location in between each call of SubVI at each cycle of program main loop – should be consistently passed from output to input with shift register

2. Logging? – set true to initialise – restarts logging with each rising edge – data type BOOL
3. Log on previous loop – required memory for "Logging?" rising edge detection – should be consistently passed from output to input with shift register – data type BOOL
4. Error in
5. Column names – Header column titles for generated ".csv" file
6. Log values – row of values to append to generated ".csv" file
7. Logging file name – user selected file name

C. Outputs

1. File path output – used to maintain generated file location in between each call of SubVI at each cycle of program main loop – should be consistently passed from output to input with shift register
2. Log for next loop – required memory for "Logging?" rising edge detection – should be consistently passed from output to input with shift register – data type BOOL
3. Error out

D. Function

1. If logging set true run outer case structure true sequence otherwise pass path and error directly to outputs, and hold output 2 false
    a. If rising edge on input 2 run nested case structure true sequence otherwise pass path and error directly through nested loop.
    Nested case structure true sequence:
        i. Obtain path to LabVIEW project file (α)
        ii. Obtain date and time and replace illegal chars ("/", ".", ":") with "_" (β)
        iii. Generate path name – "$\alpha + \backslash\text{Labview\_Logging} + \text{input } 7 + \beta + \text{". csv"}$ – and convert from string to path
        iv. Generate ".csv" file with specified path and header row given by input 5
    Outer case structure true sequence:
    b. Append input 6 to generated ".csv" file

3.5.4 "ConnectToPannelAddressSpace.vi"

A. Function

User gives desired ES number. The SubVI checks if station is occupied. If the station is available an ID derived from the local users IP address is generated and written to the holding register associated with the specific ES gate, otherwise, an error is returned, and the program halted.

B. Inputs:

1. Requested ES number – data type INT

C. Outputs:

1. Common communication cluster of:
    a. Station register start address – data type INT
    b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
    c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. Initialised error line
3. Local IP address expressed as 32-bit UINT (for diagnostics)
4. Local IP address expressed as string (for diagnostics)

D. Working

1. Obtain local ID:
    a. All system IPs are retrieved, and a for loop returns the IP address in the range associated with the server (192.168.1.0 – 192.168.1.255).
    b. IP is output as 32-bit UINT and string for potential diagnostics.
    c. Station ID is derived by taking the 16-bit modulo of the local IP address
2. Connect to server at IP 192.168.200, port 502.
3. Find address space start address – selected ES# expressed as an enumerated number (i.e.: ES1 = 0, ES2 = 1, …, ES8 = 7) and multiplied by 100.
4. Check if connection is permitted:
    a. Holding register with offset +8 (gate) is read, if entry =0, or =local ID, connection is permitted
    b. If permitted, write local ID to holding register with offset +8 (gate) and return no error.
    c. If not permitted, return error.
5. Pack register start address, local ID, and MODBUS/TCP comms line into cluster for use by proceeding communication blocks.

3.5.5 "DigitalInput.vi"

A. Function

Returns Boolean value of desired physical panel digital input. Can be daisy-chained to allow multiple reads of same input array.

B. Inputs

1. Desired read array offset – in the case of physical panel AI, this can be enumerated or int 0-7 – datatype INT
2. Array of Boolean values – can be any array, but DI is used in this project to read discrete inputs – data type nx1 array of BOOL

C. Outputs

1. Value of scalar at array offset given by input – data type BOOL
2. Identical array from SubVI input

D. Working

1. Index given by input 1 read from array given by input 2 and passed to output 1
2. Input 2 passed directly to output 2

3.5.6 "DigitalOutput.vi"

A. Function

Inserts Boolean value into 8x1 array of values to be written to output coils by write SubVI. If the 8x1 input array of values is unwired, the output array will be initialised as 8x1 array of Boolean values =false. Can be daisy-chained together to allow multiple writes to the same array.

B. Inputs

1. Desired write array offset – in the case of physical panel DO, this can be enumerated or int 0-7 – datatype INT
2. Boolean output value – data type BOOL
3. 8x1 array of values to insert output into – data type BOOL

C. Outputs

1. 8x1 array of values to pass into next "DigitalOutput.vi" SubVI OR write to MODBUS/TCP server – updated array given by input 3 – data type BOOL

D. Working

1. Boolean value given by input 2 written to array given by input 3 in position given by input 1 then passed to output 1

3.5.7 "DisconnectFromPannelAddressSpace.vi"

A. Function

Forces all physical panel analog outputs to =0 and all panel digital outputs to =false. Clears the ES gate register. If program stopped before execution, gate will be held shut until either: local user reconnects and terminates program correctly, connection times out and gate is auto-cleared, or gate is manually cleared directly in server

B. Inputs

1. Common communication cluster of:
    a. Station register start address – data type INT
    b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
    c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line in

C. Outputs

1. Common communication cluster of:
    a. Station register start address – data type INT

   b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
   c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line out

D. Working

1. Break out communication cluster to obtain local ID and ES register start address
2. Generate 8x1 array of type BOOL and value false and write to discrete coils with offset +0
3. Generate 8x1 array of type INT and value 0, and write to holding registers with offset +0
4. Write 0 to holding register with offset +8 (gate)
5. Close communication to MODBUS/TCP server and return error

3.5.8 "FieldPost.vi"

A. Function

Performs pre-cycle testing of physical I/O connection from server. LabVIEW natively handles errors because of server communication loss, however, communication between server and field I/O can fail. Server keeps watchdog for each field I/O. This SubVI checks watchdog for "CC cycles" number of communication cycles if there is "Required Pass (%)" % success rate of frame transfers cycle will begin, else, SubVI will return failure and program will halt program.

B. Inputs

1. Common communication cluster of:
   a. Station register start address – data type INT
   b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
   c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line in
3. Posting cycle count – how many times to check field I/O watch dog (optional, default 10)
4. Required pass rate – % of checked cycles watchdog must pass to pass posting (optional, default 80%)

C. Outputs

1. Common communication cluster of:
   a. Station register start address – data type INT
   b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
   c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line out

D. Working

1. Read inputs, if no user selected "Required Pass (%)" or "CC Cycles" given, take default values of 80%, and 10 checks, respectively
2. Run watchdog check "CC Cycles" times (for loop)

   a. Read MODBUS/TCP server holding register 13, corresponding to field I/O watchdog (register is updated by server at each successful transaction with device)
   b. Compare current read of watchdog to previous read (passed via shift register between cycles of loop)
   c. If current read not equal to previous read, test passes, increment count of passes
3. Compare count of passes to required count of passes (real passes $\geq \frac{\text{“Required Pass (\%)”}}{100} *$ "CC Cycles")
4. If test pass, return confirmation message and no error, else return error

3.5.9 “MaintainConnectionToPannelAddressSpace.vi”

A. Function

Cyclic rewriting of local ID to ES gate to maintain connection to requested panel.

B. Inputs

1. Common communication cluster of:
   a. Station register start address – data type INT
   b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
   c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line in

C. Outputs

1. Common communication cluster of:
   a. Station register start address – data type INT
   b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
   c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line out

D. Working

1. Break out communication cluster to obtain local ID and ES register start address
2. Write local ID to holding register with offset +8
3. Read holding register with offset +8 and output of SubVI (for diagnostics)
4. Repack communication cluster

3.5.10 “ReadFromPannelAddressSpace.vi”

A. Function

Reads all input registers, holding registers associated with scratch inputs, and discrete inputs from MODBUS/TCP server.

B. Inputs

1. Common communication cluster of:

   a. Station register start address – data type INT
   b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
   c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line in

C. Outputs

1. Common communication cluster of:
   a. Station register start address – data type INT
   b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
   c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line out
3. Input registers – 8x1 array of values associated with physical panel AI0 – AI7 – datatype UINT
4. Discrete inputs – 8x1 array of values associated with physical panel DI0 – DI7 – data type BOOL
5. Scratch input registers – 8x1 array of values associated with scratch input values from 3rd party application – datatype UINT

D. Working

1. Break out communication cluster to obtain ES register start address
2. Read width 8 input registers starting from offset +0 and return as SubVI output
3. Read width 8 holding registers starting from offset +14 and return as SubVI output
4. Read width 8 discrete inputs starting from offset +0 and return as SubVI output
5. Repack communication cluster.

3.5.11 "TimeKeeping.vi"

A. Function

Included primarily for conciseness in the main program. Manages flashing Boolean indicator on front panel to confirm continued operation. Manages program stall, program is stalled by user specified dt to maintain fixed sample time; while the program will attempt to run at interval given by input dt, server update is capped at 150ms. Tracks time since program start.

B. Inputs

1. Previous loop status – used to update flashing indicator on front panel – data type BOOL
2. Local clock value at program start – data type INT
3. Desired sample time (dt) – data type INT

C. Outputs

1. Current loop status – inverted previous loop status – data type BOOL
2. Time since program start – data type DBL

D. Working

1. Input 1 inverted and passed to output 1
2. Program stalled by dt given in input 3 output of wait block is current system clock
3. System clock at start (input 2) subtracted from current system clock to give time since start in ms
4. Time since start converted to s and passed to output 2.

3.5.12 WriteToPannelAddressSpace.vi"

A. Function

Writes all holding registers associated with physical panel analog outputs and scratch outputs, and discrete coils to server.

B. Inputs

1. Common communication cluster of:
    a. Station register start address – data type INT
    b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
    c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. Analog output array – 8x1 array of values to write to physical panel AO0 – AO7 – data type INT
3. Digital output array – 8x1 array of values to write to physical panel DO0-DO7 – data type BOOL
4. Scratch output array – 8x1 array of values to write to holding registers associated with inter-application communication.
5. error line out

C. Outputs

1. Common communication cluster of:
    a. Station register start address – data type INT
    b. User ID – 16-bit modulo of local user IP expressed as a 32-bit integer – datatype INT
    c. MODBUS/TCP communication line – LabVIEW common MODBUS/TCP support
2. error line out

D. Working

1. All 8x1 input arrays (input 2 – AO, input 3 – DO, input 4 – scratch outputs) are forced into 8x1 array to prevent dimension mismatch errors
2. Break out communication cluster to obtain ES register start address
3. Write 8x1 array of DO with width 8 to discrete coils starting from offset +0
4. Write 8x1 array of AO with width 8 to holding registers starting from offset +0
5. Write 8x1 array of scratch outputs with width 8 to holding registers starting from offset +24
6. Repack communication cluster

APPENDIX A. SubVIs

This appendix contains figures of Client Template supporting LabVIEW subVI's, made up of the following:

1. "AnalogInput.vi"
2. "AnalogOutput.vi"
3. "arbitrarySessionLog.vi"
4. "ConnectToPannelAddressSpace.vi"
5. "DigitalInput.vi"
6. "DigitalOutput.vi"
7. "DisconnectFromPannelAddressSpace.vi"
8. "FieldPost.vi"
9. "MaintainConnectionToPannelAddressSpace.vi"
10. "ReadFromPannelAddressSpace.vi"
11. "TimeKeeping.vi"
12. "WriteToPannelAddressSpace.vi"

For descriptions of SubVI function see preceding sections.

A1. "AnalogInput.vi"

Subvi Front Panel:

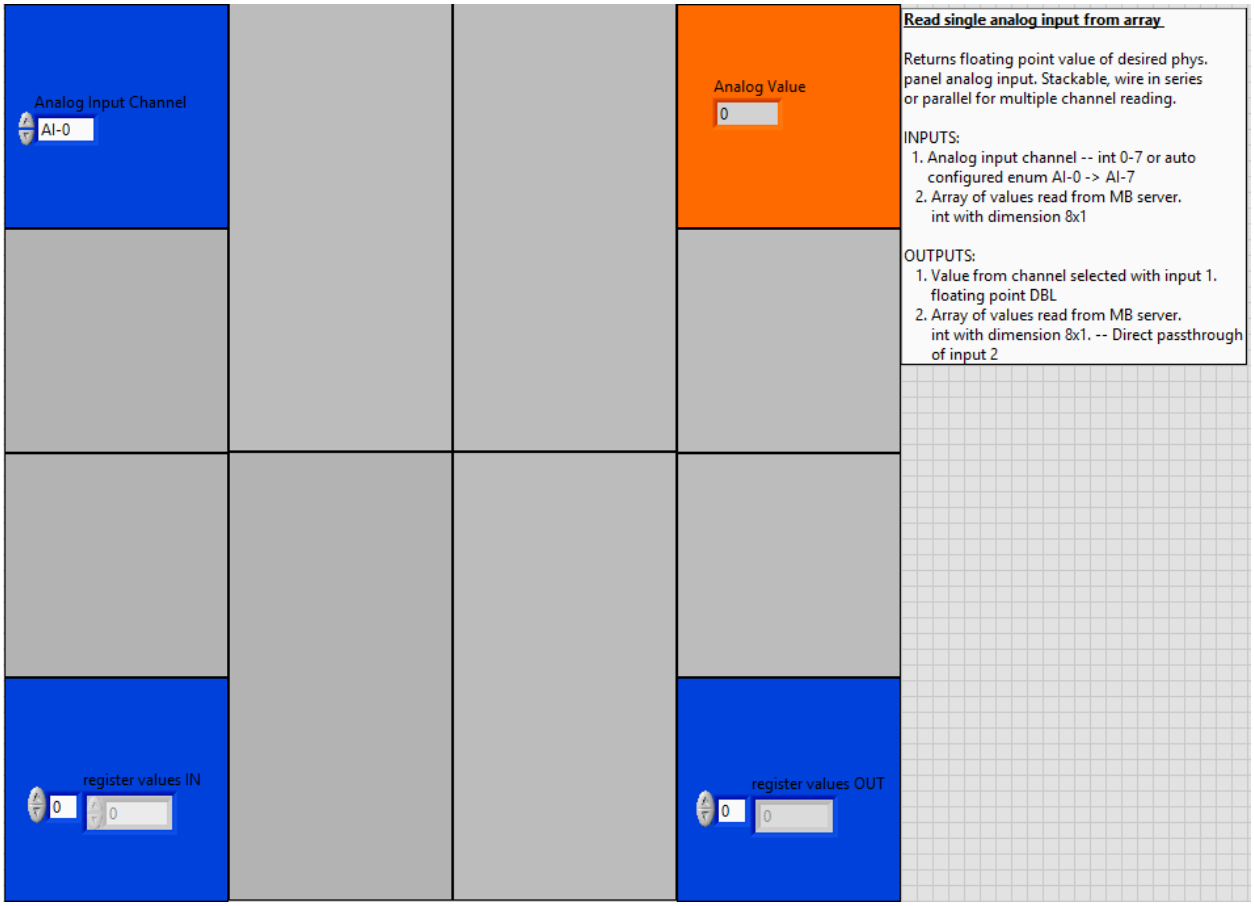


Figure APX 1: "AnalogInput.vi" LabVIEW front panel.

Subvi Back Panel:

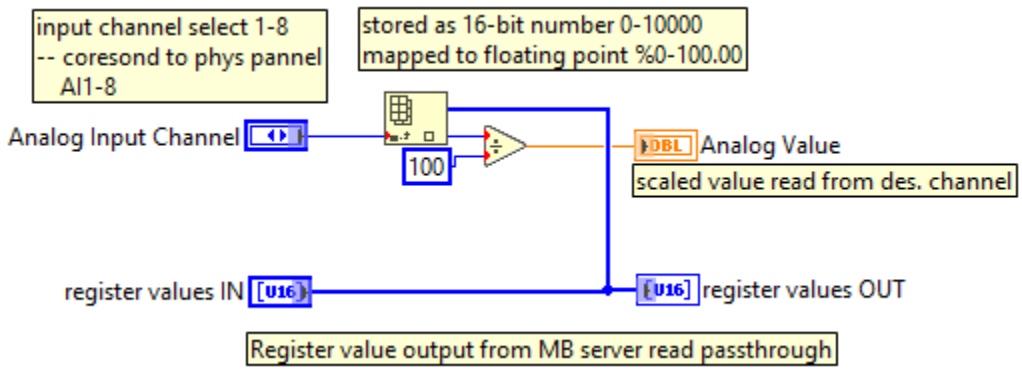


Figure APX 2: "AnalogInput.vi" LabVIEW back panel.

A2. "AnalogOutput.vi"

Subvi Front Panel:

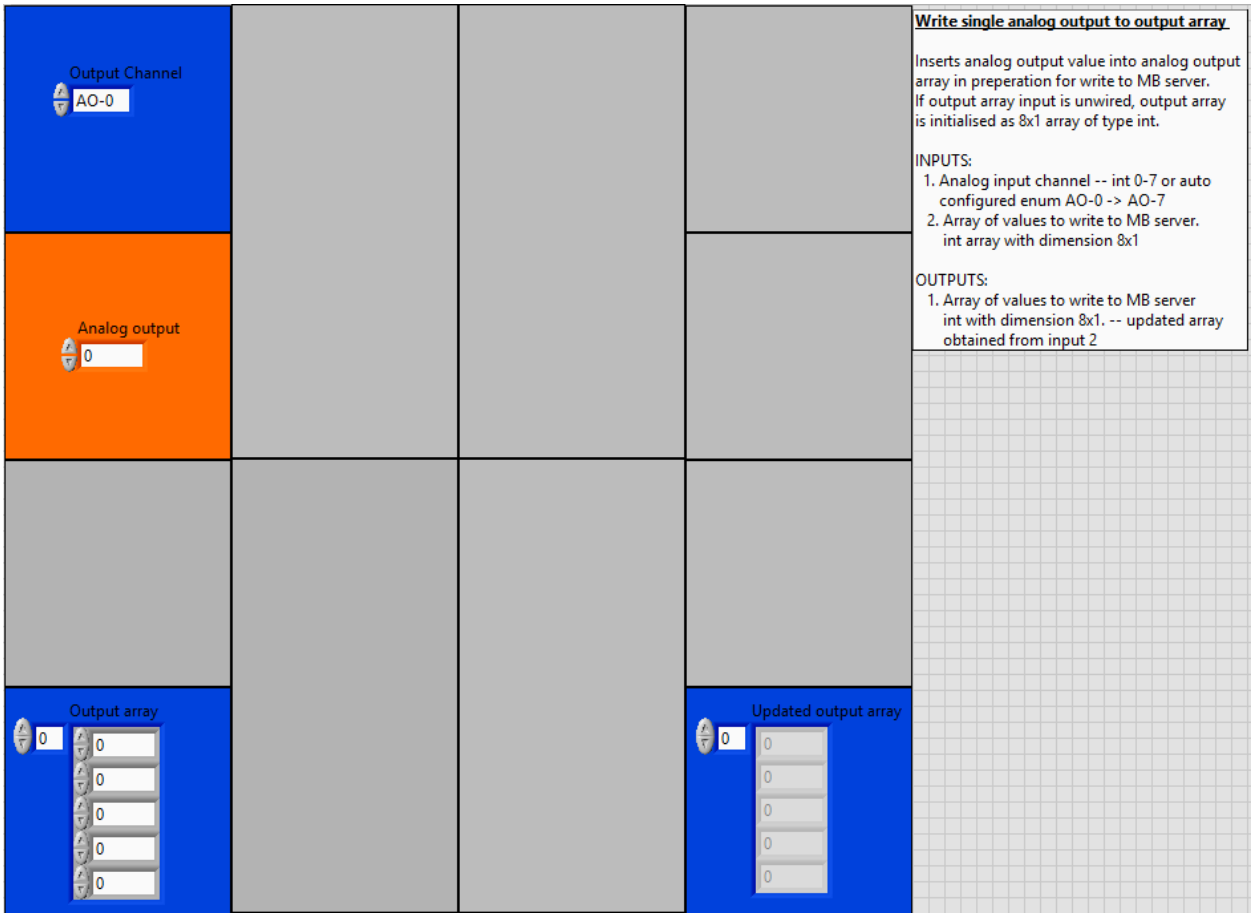


Figure APX 3: "AnalogOutput.vi" LabVIEW front panel.

Subvi Back Panel:

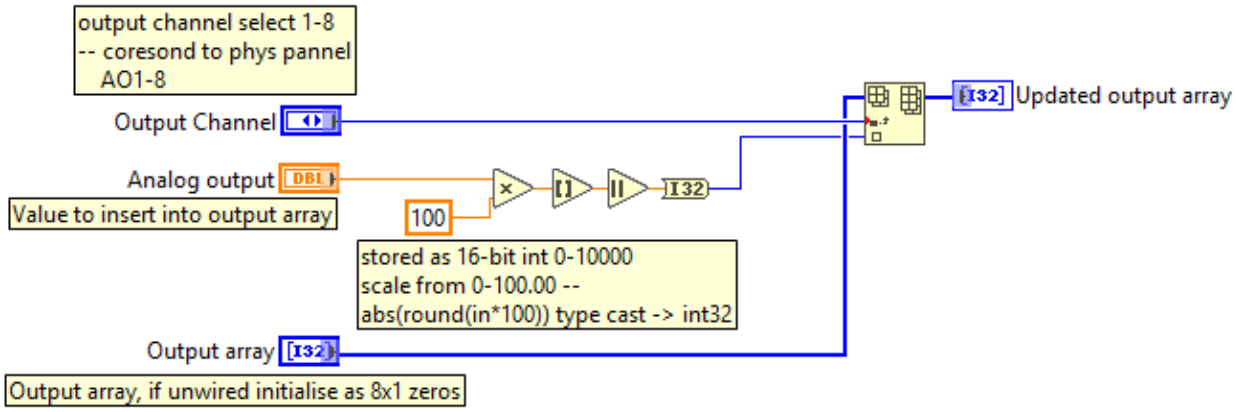


Figure APX 4: "AnalogOutput.vi" LabVIEW back panel.

A3. "arbitrarySessionLog.vi"

Subvi Front Panel:

Figure APX 5: "arbitrarySessionLog.vi" LabVIEW front panel.

Subvi Back Panel:

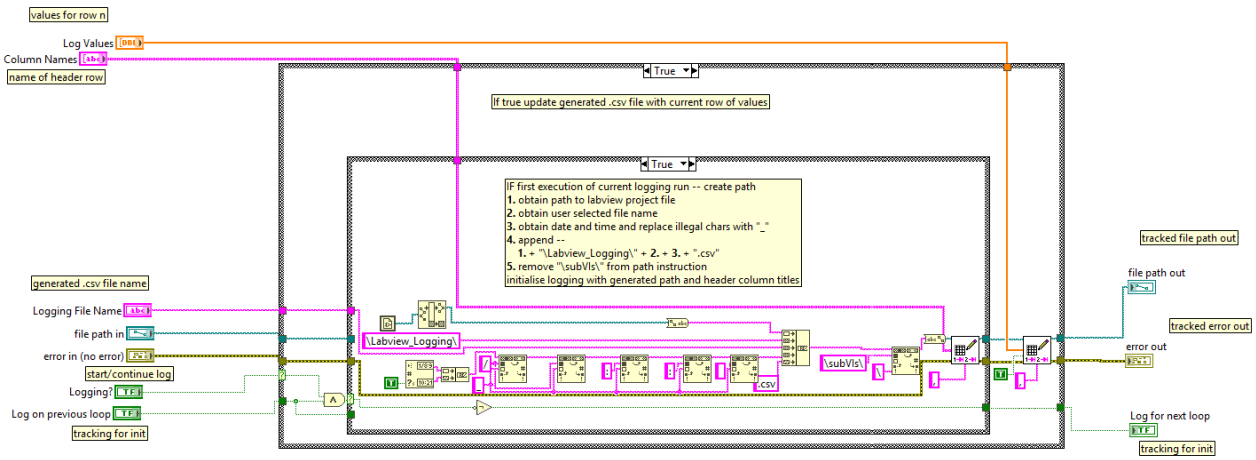


Figure APX 6: "arbitrarySessionLog.vi" LabView back panel.

A4. "ConnectToPannelAddressSpace.vi"

Subvi Front Panel:

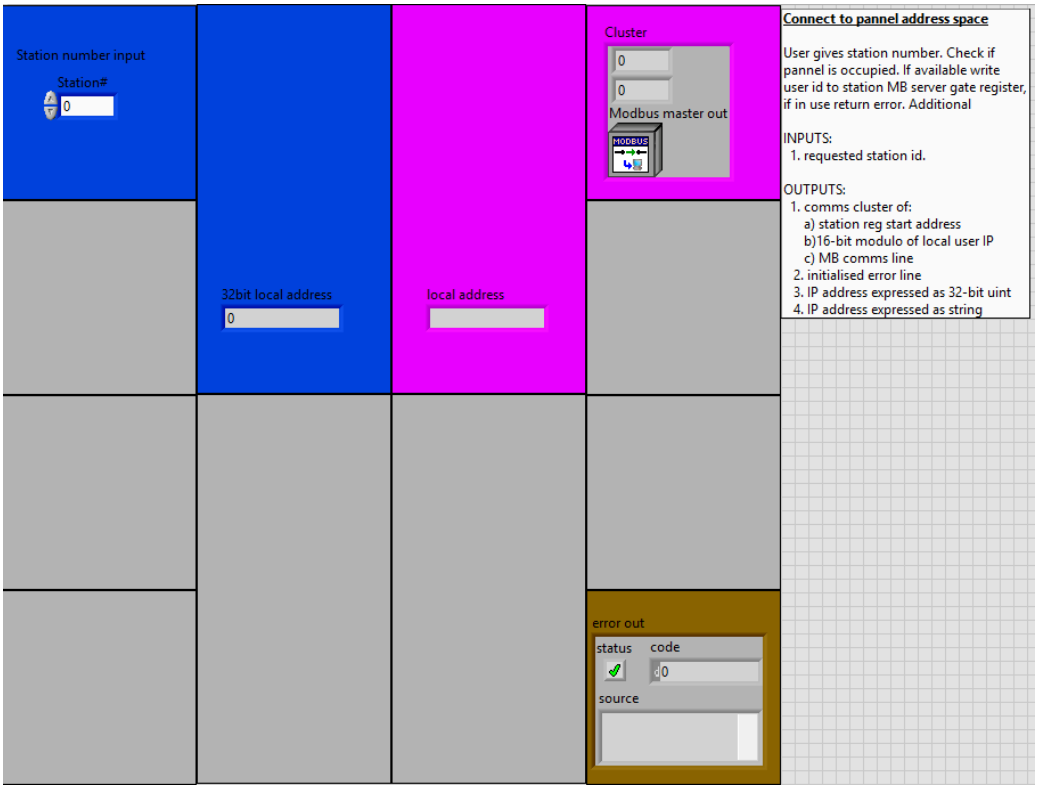


Figure APX 7: "ConnectToPannelAddressSpace.vi" LabVIEW front panel.

Subvi Back Panel:

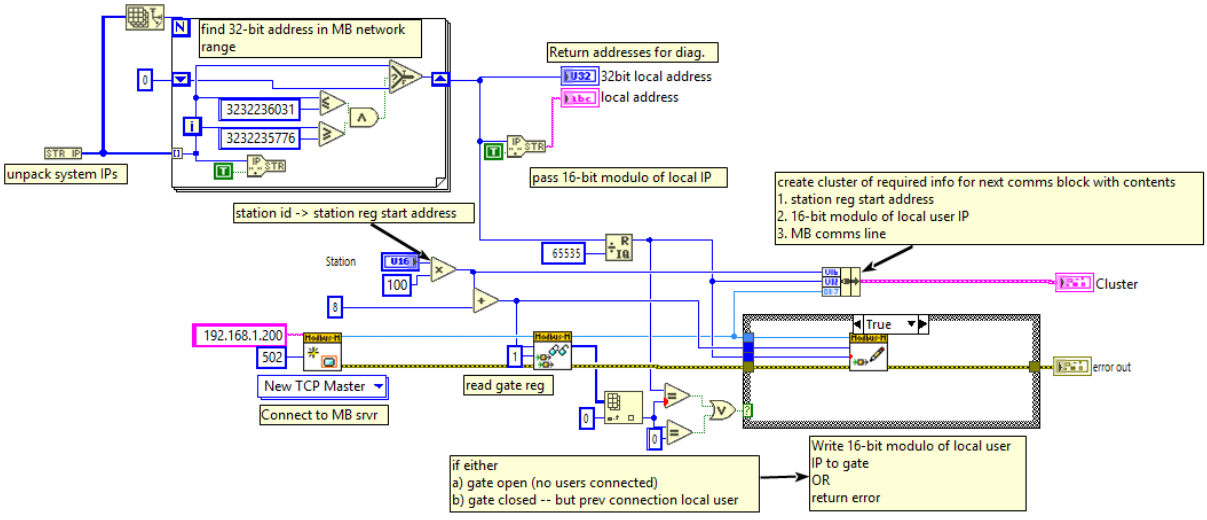


Figure APX 8: "ConnectToPannelAddressSpace.vi" LabVIEW back panel.

A5. "DigitalInput.vi"

Subvi Front Panel:

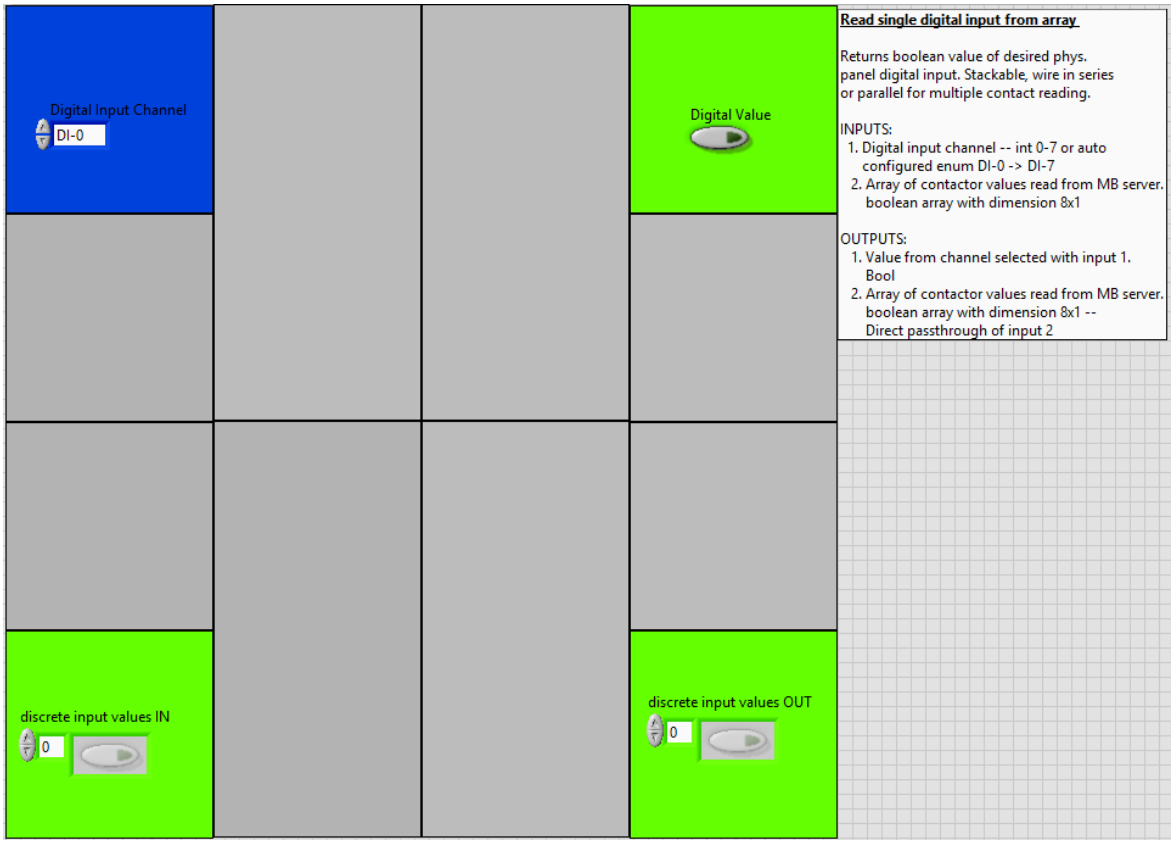


Figure APX 9: "DigitalInput.vi" LabVIEW front panel.

Subvi Back Panel:

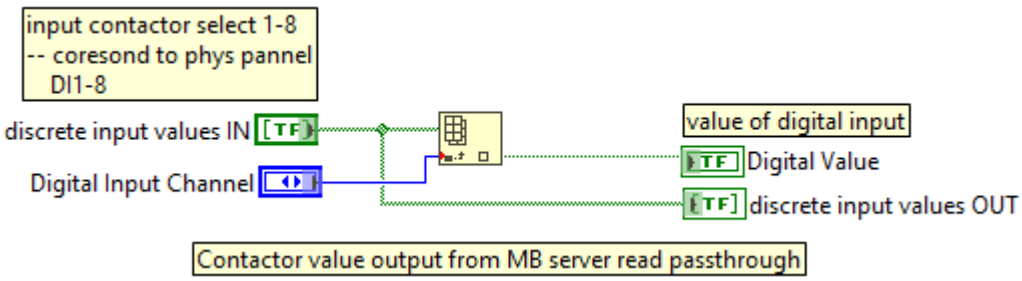


Figure APX 10: "DigitalInput.vi" LabVIEW front panel.

A6. "DigitalOutput.vi"

Subvi Front Panel:

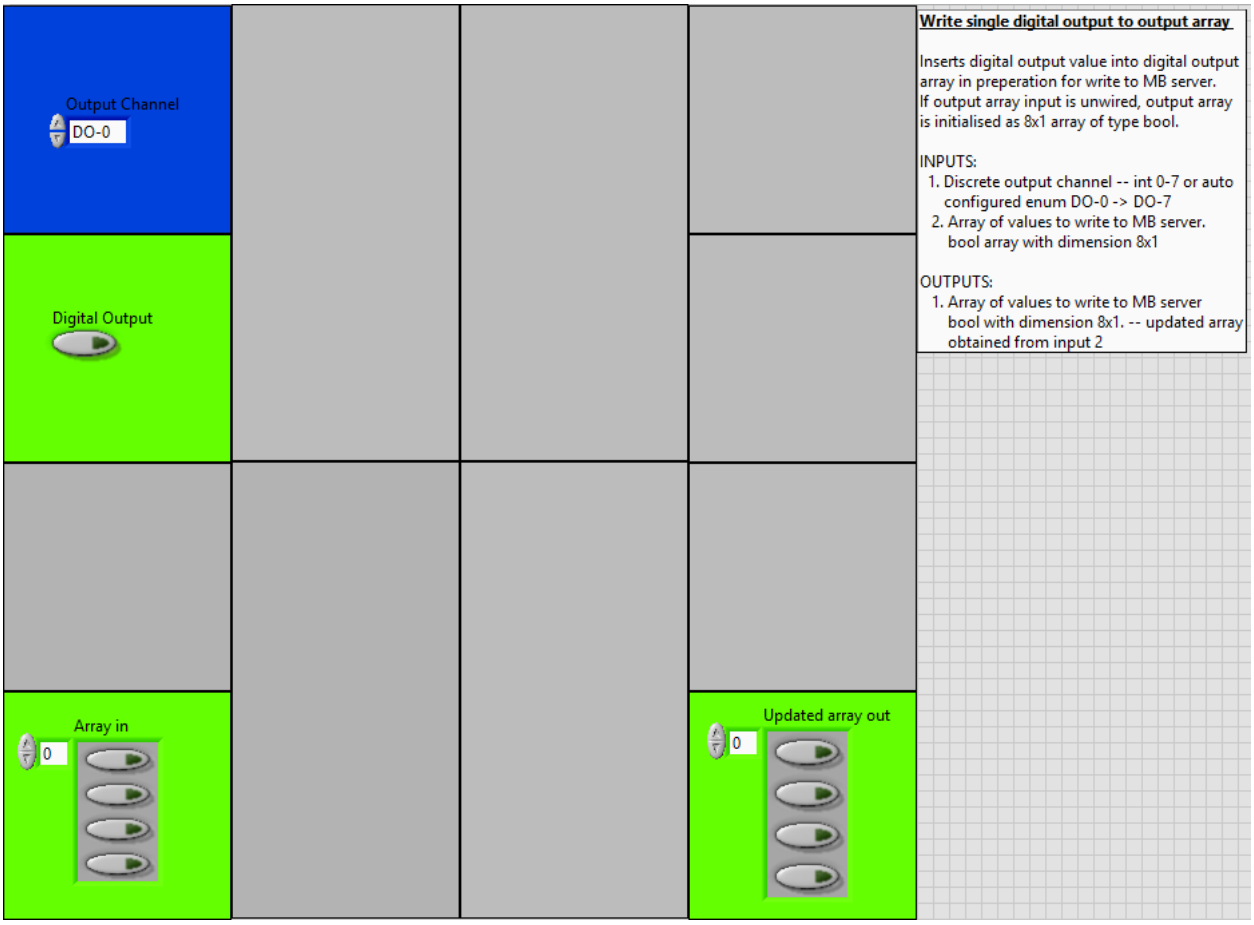


Figure APX 11: "DigitalOutput.vi" LabVIEW front panel.

Subvi Back Panel:

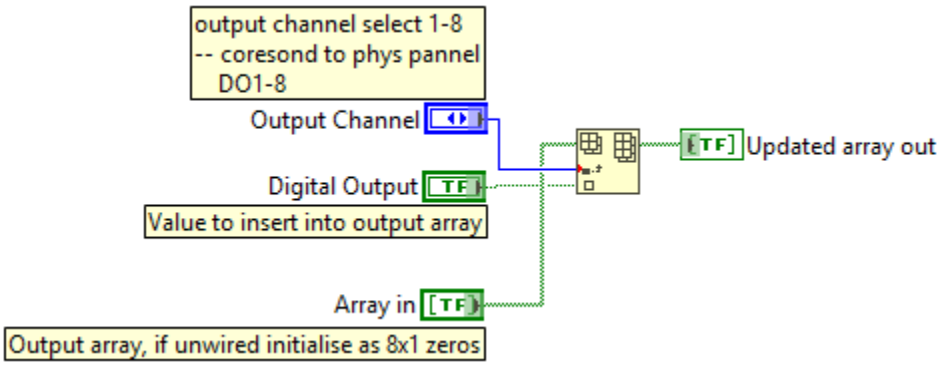


Figure APX 12: "DigitalOutput.vi" LabVIEW back panel.

A7. "DisconnectFromPannelAddressSpace.vi"

Subvi Front Panel:

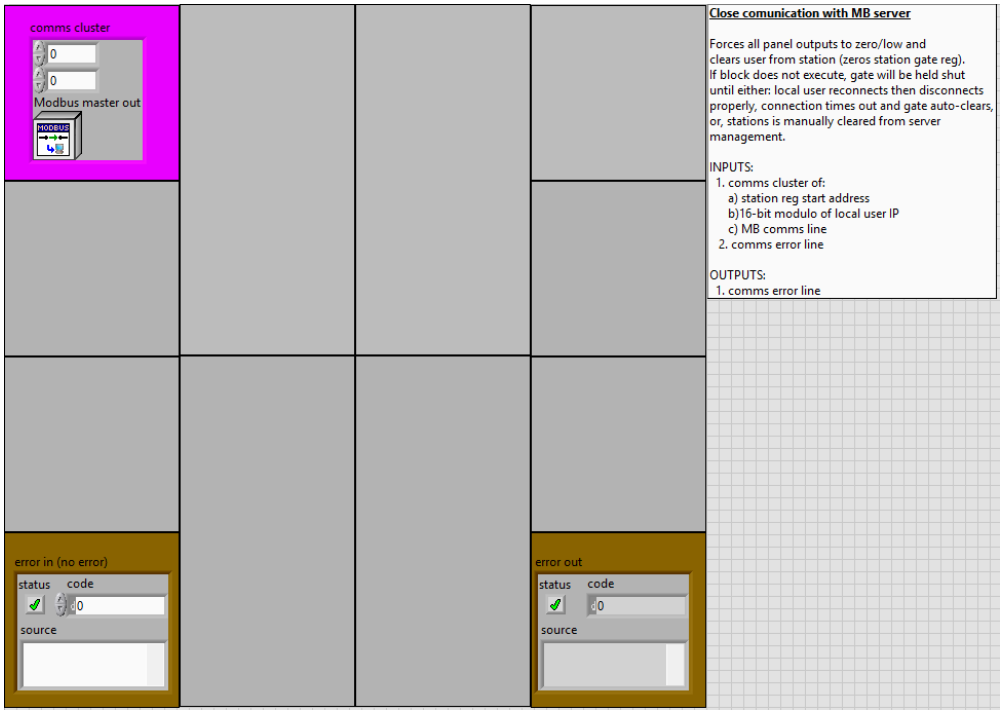


Figure APX 13: "DisconnectFromPannelAddressSpace.vi" LabVIEW front panel.

Subvi Back Panel:

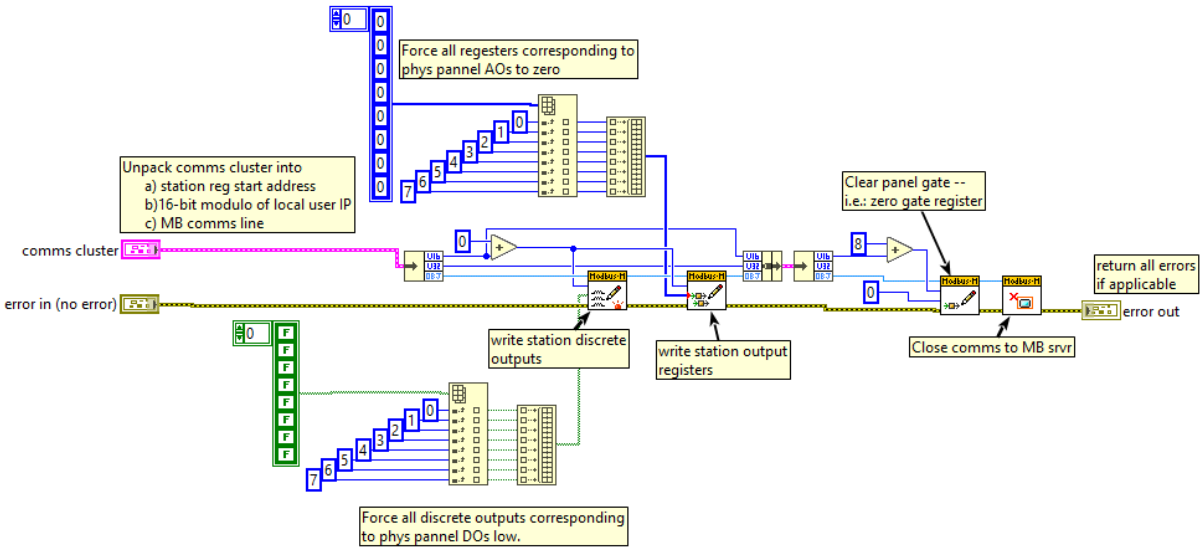
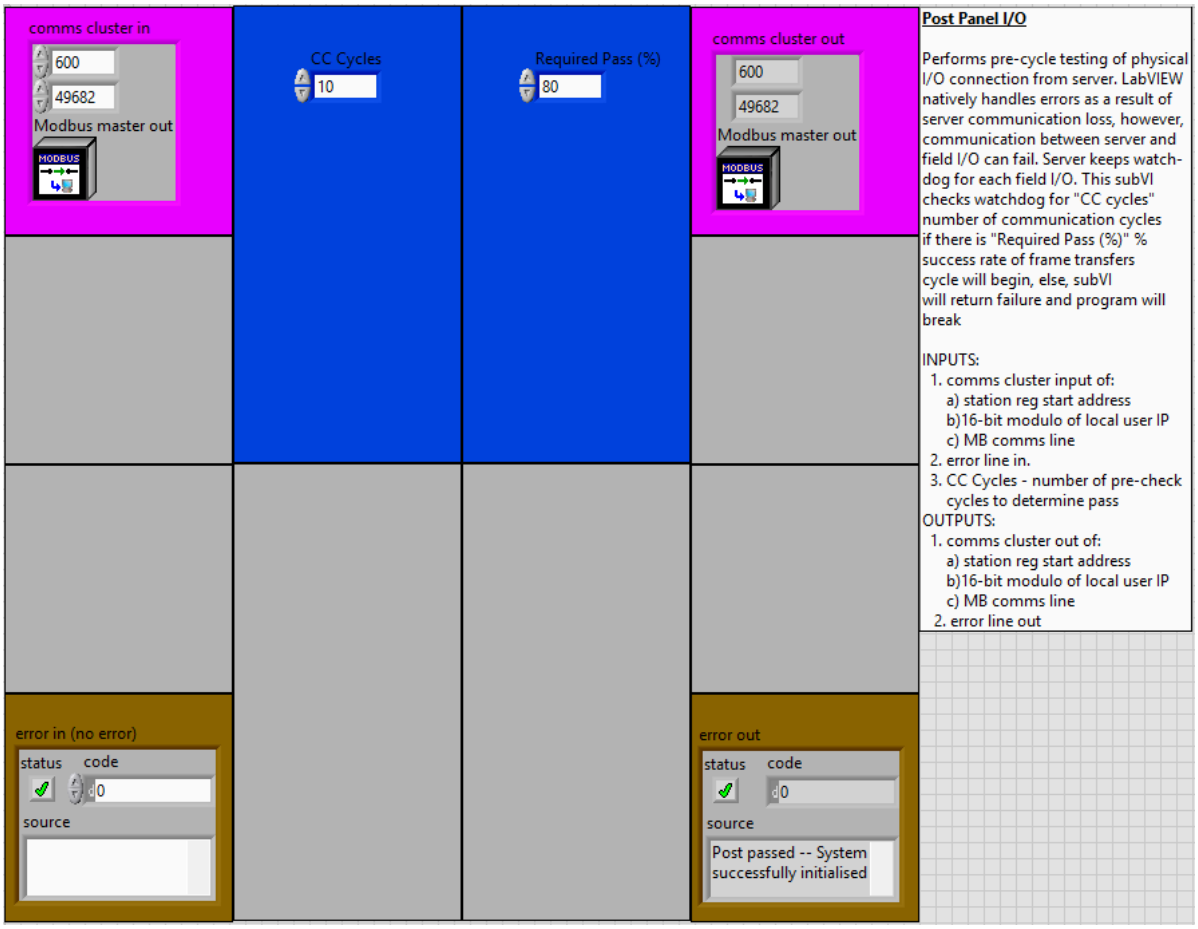


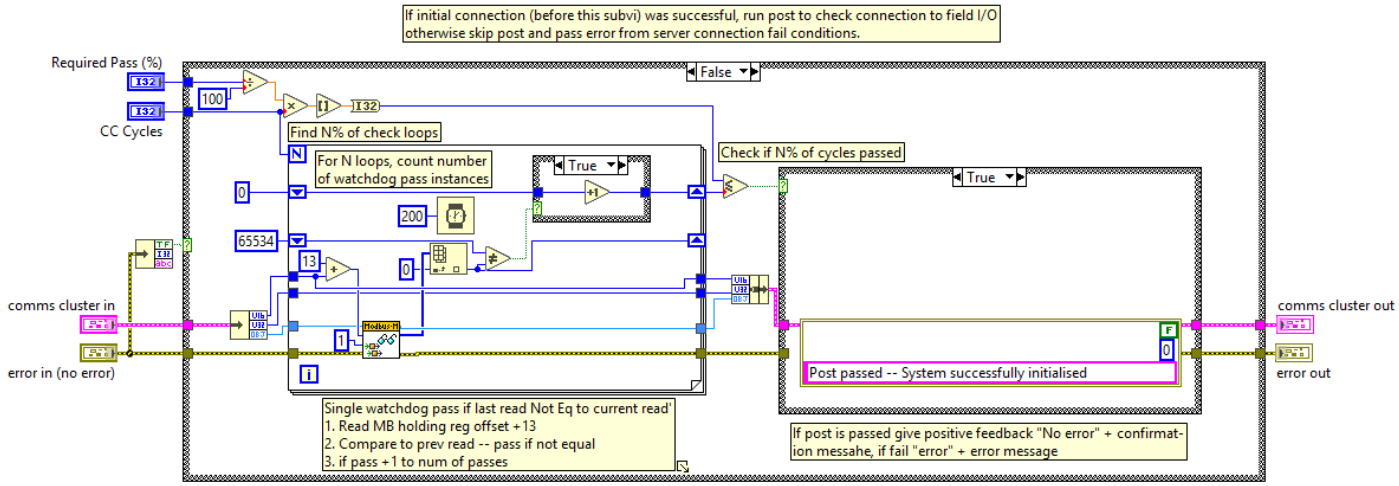
Figure APX 14: "DisconnectFromPannelAddressSpace.vi" LabVIEW back panel.

A8. "FieldPost.vi"

SubVI front panel:



SubVI back panel:



A9. "MaintainConnectionToPannelAddressSpace.vi"

Subvi Front Panel:

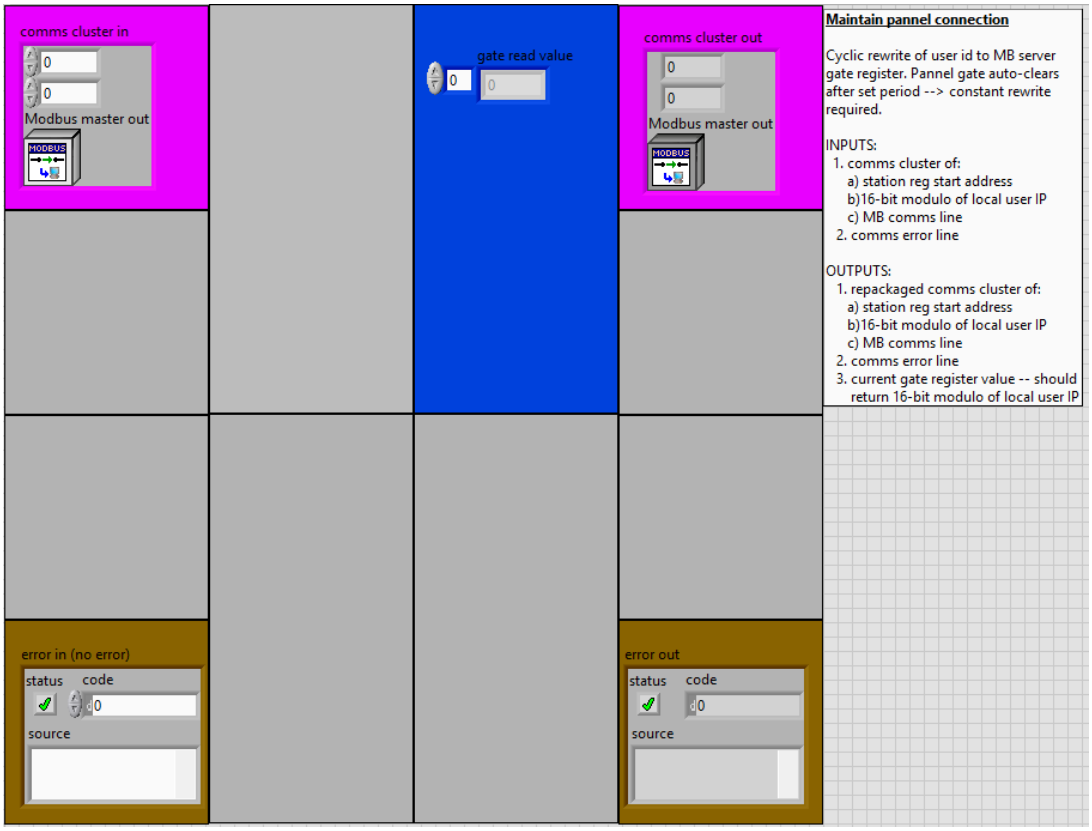


Figure APX 15: "MaintainConnectionToPannelAddressSpace.vi" LabVIEW front panel.

Subvi Back Panel:

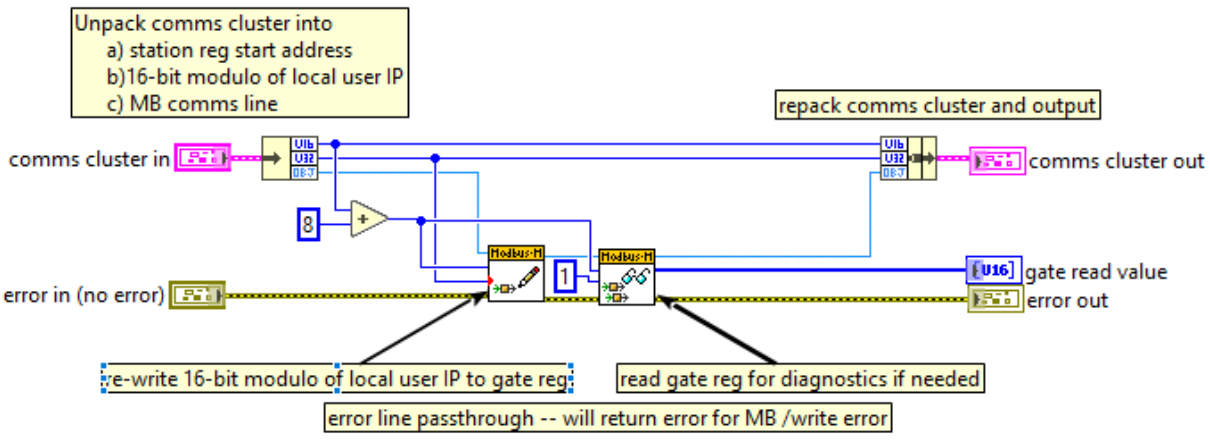


Figure APX 16: "MaintainConnectionToPannelAddressSpace.vi" LabVIEW back panel.

A10. "ReadFromPannelAddressSpace.vi"

Subvi Front Panel:

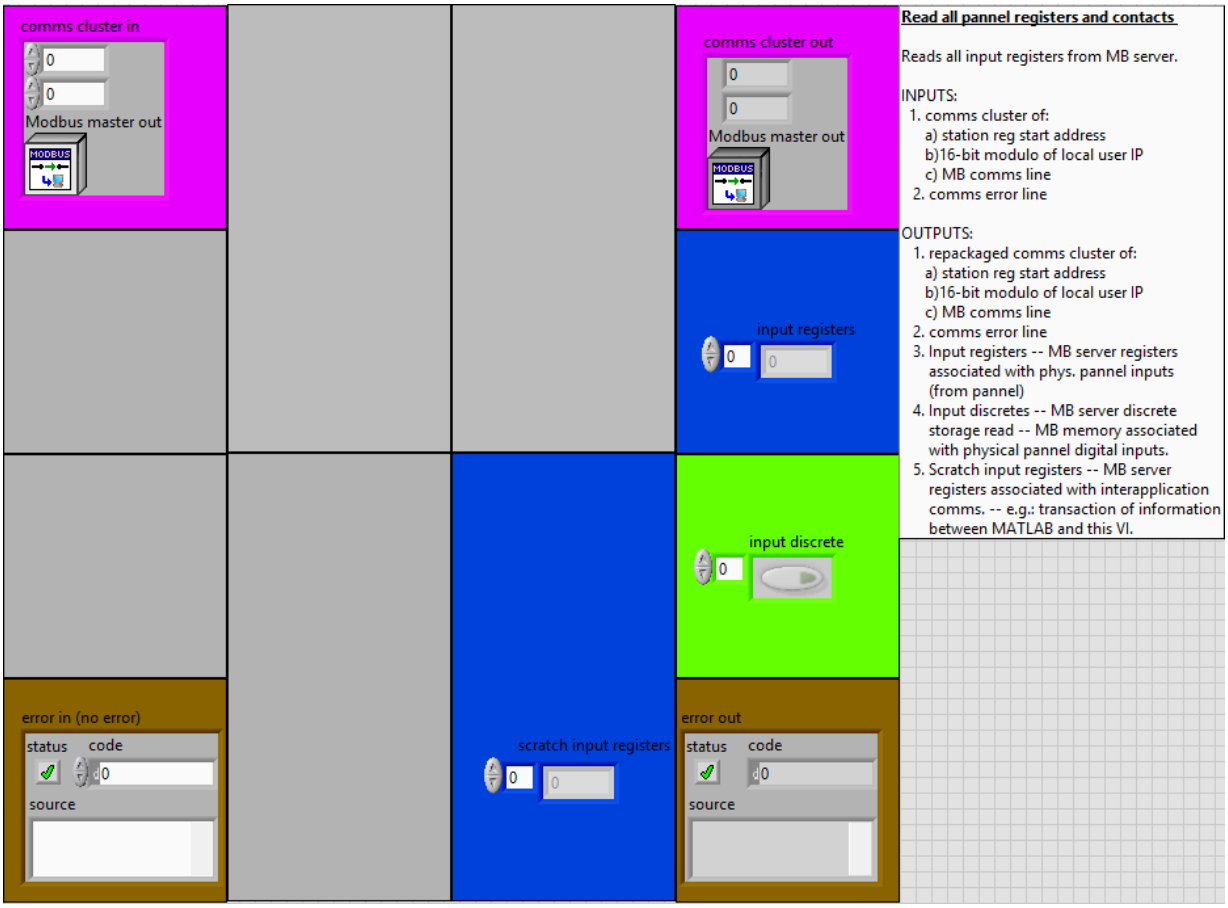


Figure APX 17: "ReadFromPannelAddressSpace.vi" LabVIEW front panel.

Subvi Back Panel:

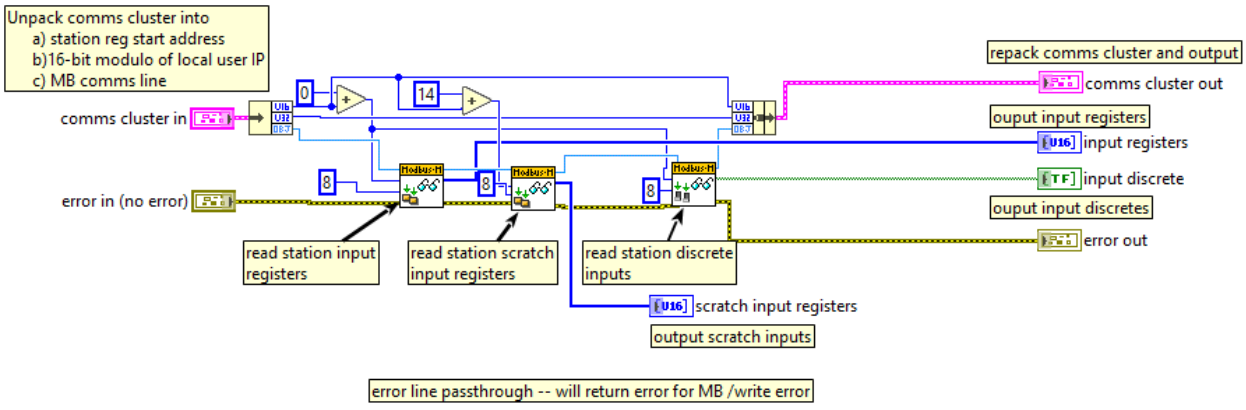


Figure APX 18: "ReadFromPannelAddressSpace.vi" LabVIEW back panel.

A11. "TimeKeeping.vi"

Subvi Front Panel:

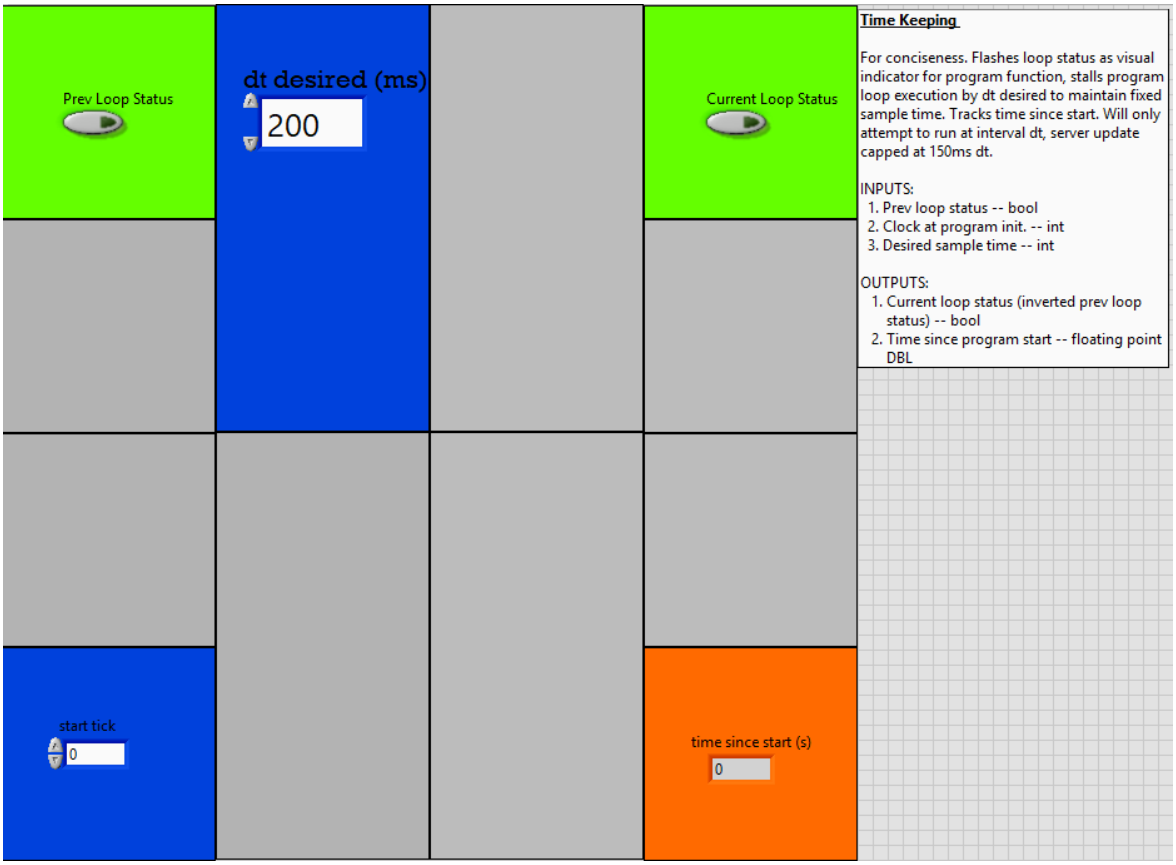


Figure APX 19: "TimeKeeping.vi" LabVIEW front panel.

Subvi Back Panel:

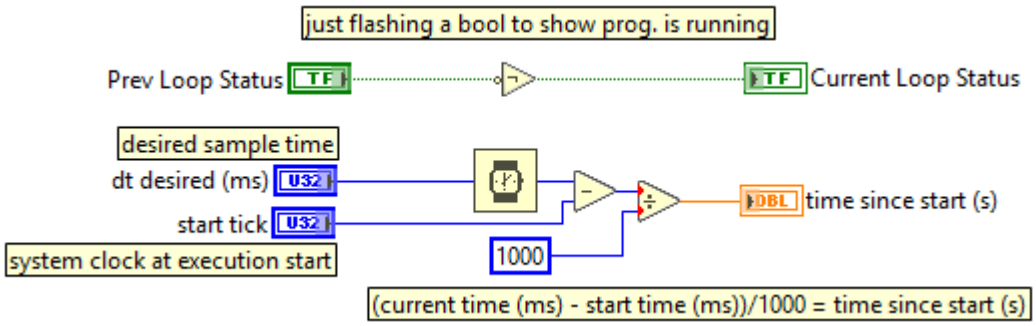


Figure APX 20: "TimeKeeping.vi" LabVIEW back panel.

A12. "WriteToPannelAddressSpace.vi"

Subvi Front Panel:

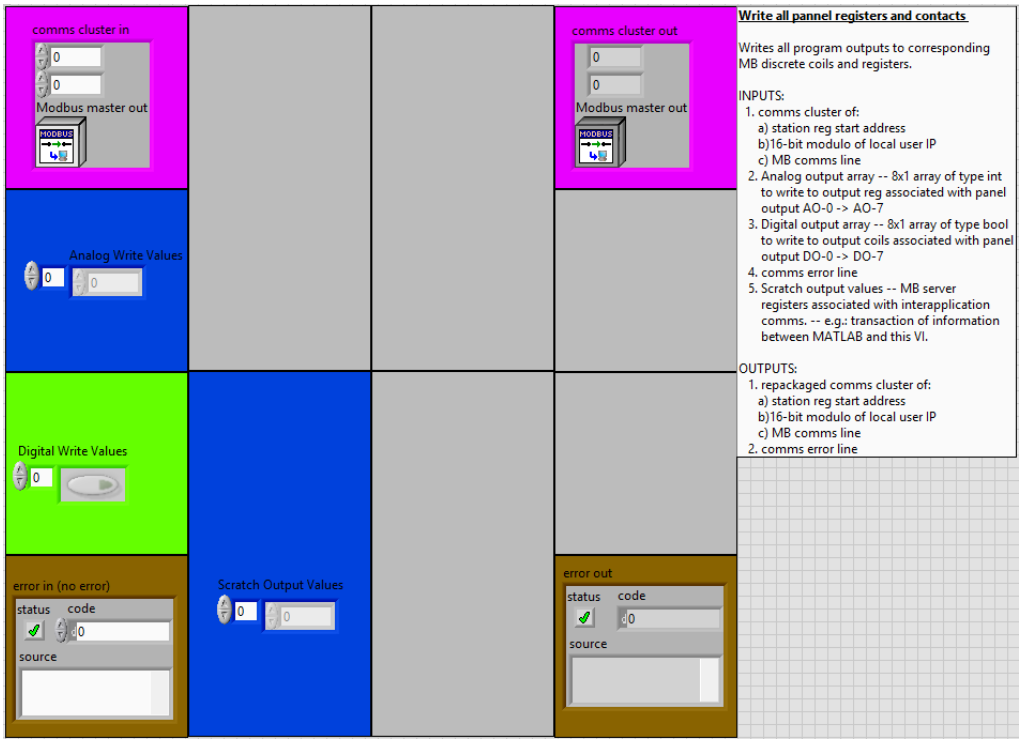


Figure APX 21: "WriteToPannelAddressSpace.vi" LabVIEW front panel.

Subvi Back Panel:

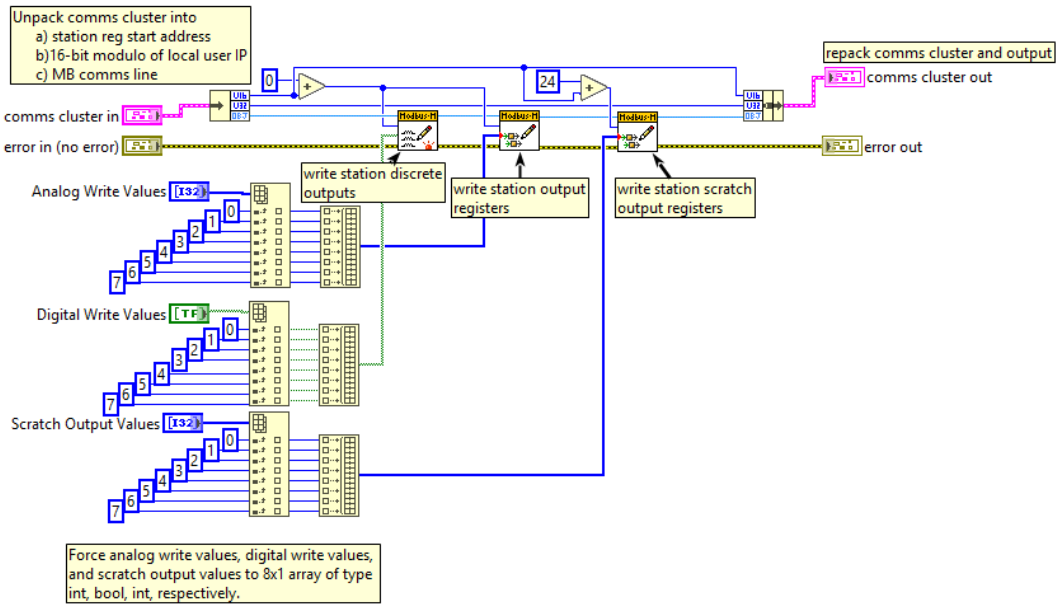


Figure APX 22: "WriteToPannelAddressSpace.vi" LabVIEW back panel.

Appendix B. 3rd Party Program Interface

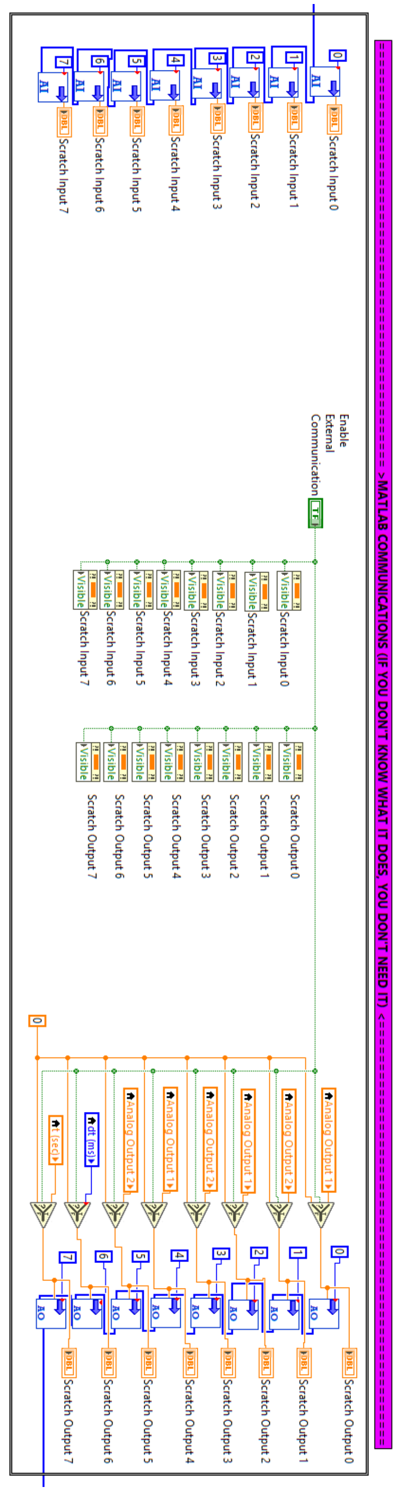


Figure APX 23: Portion of client template main loop responsible for coordination of communication between LabVIEW and 3rd party applications for the purposes of offloading complex control strategies.